Table of Contents

..... 1
========== CONFIGURATION ========== 1
========== FILE SELECTION ========== 1
========== LOAD AND PROCESS DATA ========== 2
--- DETERMINE SAMPLING FREQUENCY --- 2
--- EXTRACT GLOBAL TRACE --- 2
--- BANDPASS FILTERING --- 3
--- POLARITY CORRECTION --- 3
--- Z-SCORE NORMALIZATION --- 3
--- CREATE TIME AXIS --- 3
--- GRID EXTRACTION (6x6) --- 3
========== CREATE INTERACTIVE VIEWER ========== 5
--- LAYOUT DEFINITIONS --- 5
--- SPATIAL MAP (Right side) --- 6
--- CONTROLS --- 7
========== CALLBACK FUNCTIONS ========== 7


```
% Combined_Signal_Viewer_6x6.m
% Signal Viewer with 6x6 Grid Decomposition
%
% FEATURES:
%   - Automatically splits the spatial map into a 6x6 grid (36 regions).
%   - Extracts and Z-scores signals for all 36 regions.
%   - Interactive Viewer with dynamic 6x6 trace grid layout.

clear; clc; close all;
```

========== CONFIGURATION ==========

```
CONFIG.bandpass_freq = [0.1, 75];     % Hz - Bandpass filter range
CONFIG.filter_order = 4;              % Butterworth filter order
CONFIG.default_fs = 500;              % Hz - Default sampling frequency
CONFIG.grid_rows = 6;                 % UPDATED: 6 Rows
CONFIG.grid_cols = 6;                 % UPDATED: 6 Columns
```

========== FILE SELECTION ==========

```
fprintf('=======================================\n');
fprintf(' Signal Viewer - 6x6 Grid Mode\n');
fprintf('=======================================\n\n');
fprintf('Please select your reconstruction file...\n');

[filename, pathname] = uigetfile('*.mat', 'Select Reconstruction File');
```

```
if isequal(filename, 0)
    fprintf('Selection canceled by user.\n');
    return;
end
full_path = fullfile(pathname, filename);
fprintf('Processing: %s\n', filename);

======================================
 Signal Viewer - 6x6 Grid Mode
======================================

Please select your reconstruction file...
Selection canceled by user.
```

========== LOAD AND PROCESS DATA ==========

```
try

    % Load data
    D = load(full_path, 'reconstructed_movie', 'Fs');

    if ~isfield(D, 'reconstructed_movie')
        error('No reconstructed_movie field found in file.');
    end

    mov = D.reconstructed_movie;
    [height, width, num_frames] = size(mov);
    fprintf('  Dimensions: %d x %d x %d frames\n', height, width,
num_frames);
```

--- DETERMINE SAMPLING FREQUENCY ---

```
    if isfield(D, 'Fs')
        Fs = D.Fs;
    else
        tok = regexp(filename, '_(\d+)_', 'tokens', 'once');
        if ~isempty(tok)
            Fs = str2double(tok{1});
        else
            Fs = CONFIG.default_fs;
        end
    end
    fprintf('  Sampling rate: %d Hz\n', Fs);
```

--- EXTRACT GLOBAL TRACE ---

```
    raw_trace = squeeze(mean(mean(mov, 1), 2));
```

--- BANDPASS FILTERING ---

```
    nyq = Fs / 2;
    [b, a] = butter(CONFIG.filter_order, CONFIG.bandpass_freq / nyq,
'bandpass');
    clean_trace = filtfilt(b, a, raw_trace);
```

--- POLARITY CORRECTION ---

```
    half_idx = floor(num_frames / 2);
    second_half_trace = clean_trace(half_idx:end);
    sk_2nd_half = skewness(second_half_trace);

    if sk_2nd_half < 0
        curr_trace = -clean_trace;
        polarity_action = 'FLIPPED';
    else
        curr_trace = clean_trace;
        polarity_action = 'KEPT';
    end
```

--- Z-SCORE NORMALIZATION ---

```
    mu = mean(curr_trace);
    sigma = std(curr_trace);

    if sigma ~= 0
        final_trace = (curr_trace - mu) / sigma;
    else
        final_trace = curr_trace;
    end
```

--- CREATE TIME AXIS ---

```
    time_axis = (0:length(final_trace)-1) / Fs;

    fprintf('  ✓ Global processing complete\n');
```

--- GRID EXTRACTION (6x6) ---

```
    fprintf('  Processing %dx%d Grid (36 Regions)...\n', CONFIG.grid_rows,
CONFIG.grid_cols);

    ROIs = struct();
    block_h = floor(height / CONFIG.grid_rows);
    block_w = floor(width / CONFIG.grid_cols);

    % Pre-calculate filter vars to speed up loop
    mov_reshaped = reshape(mov, [], num_frames);
```

```
    idx_counter = 0;
    for r = 1:CONFIG.grid_rows
        for c = 1:CONFIG.grid_cols
            idx_counter = idx_counter + 1;

            % Define boundaries (handle edge cases for last row/col)
            r_start = (r-1)*block_h + 1;
            if r == CONFIG.grid_rows, r_end = height; else, r_end =
r*block_h; end

            c_start = (c-1)*block_w + 1;
            if c == CONFIG.grid_cols, c_end = width; else, c_end =
c*block_w; end

            % Create mask
            mask = false(height, width);
            mask(r_start:r_end, c_start:c_end) = true;

            % Store ROI Metadata
            ROIs(idx_counter).row = r;
            ROIs(idx_counter).col = c;
            ROIs(idx_counter).mask = mask;
            ROIs(idx_counter).pos = [c_start, r_start, (c_end-c_start),
(r_end-r_start)]; % [x,y,w,h]

            % Extract Trace
            mask_indices = find(mask);
            if ~isempty(mask_indices)
                roi_raw = mean(mov_reshaped(mask_indices, :), 1)';
            else
                roi_raw = zeros(num_frames, 1);
            end

            % Filter & Normalize
            roi_clean = filtfilt(b, a, roi_raw);
            if strcmp(polarity_action, 'FLIPPED')
                roi_clean = -roi_clean;
            end

            roi_mu = mean(roi_clean);
            roi_sigma = std(roi_clean);

            if roi_sigma ~= 0
                ROIs(idx_counter).trace = (roi_clean - roi_mu) / roi_sigma;
            else
                ROIs(idx_counter).trace = roi_clean;
            end

            % Progress indicator
            if mod(idx_counter, CONFIG.grid_cols) == 0, fprintf('.'); end
        end
    end
    fprintf('\n  ✓ %dx%d Grid extracted\n', CONFIG.grid_rows,
```

```
CONFIG.grid_cols);
    fprintf('=====================================\n\n');

catch ME
    fprintf('  ✗ ERROR: %s\n', ME.message);
    return;
end
```

========== CREATE INTERACTIVE VIEWER ==========

```
fprintf('Launching interactive viewer...\n');
h = figure('Name', ['Signal Viewer 6x6 - ' filename], ...
           'Position', [50, 50, 1600, 900], ...
           'Color', 'w', 'NumberTitle', 'off', 'MenuBar', 'none');

% Initialize viewer data structure
V = struct();
V.mObj = matfile(full_path);
V.filename = filename;
V.Fs = Fs;
V.num_frames = num_frames;
V.trace = final_trace;
V.time_axis = time_axis;
V.polarity = polarity_action;
V.curr_frame = 1;
V.ROIs = ROIs;
V.grid_rows = CONFIG.grid_rows;
V.grid_cols = CONFIG.grid_cols;
```

--- LAYOUT DEFINITIONS ---

1. MAIN GLOBAL TRACE

```
V.ax_main = axes('Parent', h, 'Position', [0.05, 0.85, 0.90, 0.12]);
plot(time_axis, final_trace, 'k-', 'LineWidth', 1.2);
hold on;
V.xline_main = xline(time_axis(1), 'r-', 'LineWidth', 2);
title('Full Field Trace', 'FontSize', 11, 'FontWeight', 'bold');
ylabel('Z-Score'); axis tight; grid on;
set(V.ax_main, 'ButtonDownFcn', @(s,e) traceClick(s,e,h));

% 2. GRID TRACES (Dynamic Layout)
grid_area_x = 0.05;
grid_area_y = 0.05;
grid_area_w = 0.45;
grid_area_h = 0.70;

% Tighter gap for 6x6
gap = 0.003;
num_plots = V.grid_rows * V.grid_cols;
w_sub = (grid_area_w - ((V.grid_cols-1)*gap)) / V.grid_cols;
```

```
h_sub = (grid_area_h - ((V.grid_rows-1)*gap)) / V.grid_rows;

V.grid_axes = gobjects(num_plots,1);
V.grid_xlines = gobjects(num_plots,1);

for i = 1:num_plots
    r = ROIs(i).row;
    c = ROIs(i).col;

    % Calculate position (Matrix coordinates: Row 1 is top)
    pos_x = grid_area_x + (c-1)*(w_sub + gap);
    pos_y = (grid_area_y + grid_area_h) - r*h_sub - (r-1)*gap;

    ax = axes('Parent', h, 'Position', [pos_x, pos_y, w_sub, h_sub]);
    plot(time_axis, ROIs(i).trace, 'b-', 'LineWidth', 0.5); % Very thin line
for 36 plots
    hold on;
    V.grid_xlines(i) = xline(time_axis(1), 'k-', 'LineWidth', 1.0);

    % Minimal styling
    set(ax, 'XTickLabel', [], 'YTickLabel', [], 'Box', 'on');
    grid on;
    axis tight;
    ylim([-3 3]);

    set(ax, 'ButtonDownFcn', @(s,e) traceClick(s,e,h));
    V.grid_axes(i) = ax;
end

% Add label
uicontrol(h, 'Style','text', 'String', '6x6 Spatial Grid Traces', ...
    'Units', 'normalized', 'Position', [grid_area_x,
grid_area_y+grid_area_h+0.005, grid_area_w, 0.03], ...
    'BackgroundColor', 'w', 'FontWeight', 'bold');
```

--- SPATIAL MAP (Right side) ---

```
V.ax_movie = axes('Parent', h, 'Position', [0.55, 0.25, 0.40, 0.50]);

frame1 = V.mObj.reconstructed_movie(:,:,1);
if strcmp(polarity_action, 'FLIPPED'), frame1 = -frame1; end

V.img_h = imagesc(frame1);
axis image off;
colormap jet;
colorbar;
hold on;

% Draw Grid Overlay
for i = 1:num_plots
    pos = ROIs(i).pos; % [x, y, w, h]
    rectangle('Position', pos, 'EdgeColor', [0.5 0.5 0.5], 'LineWidth', 0.5,
'LineStyle', ':');
```

```
end
% Border
rectangle('Position', [1 1 width height], 'EdgeColor', 'k', 'LineWidth', 2);

V.title_h = title('Frame 1 / Time: 0.00s', 'FontSize', 12);
```

--- CONTROLS ---

```
panel_filt = uipanel('Parent', h, 'Title', 'Controls', ...
                     'Position', [0.55, 0.05, 0.40, 0.15], ...
                     'BackgroundColor', 'w');

V.chk_blur = uicontrol('Parent', panel_filt, 'Style', 'checkbox', ...
                       'String', 'Spatial Blur', ...
                       'Units', 'normalized', 'Position', [0.05, 0.6, 0.3,
0.3], ...
                       'BackgroundColor', 'w', ...
                       'Callback', @(s,e) updateFrame(h, V.curr_frame));

V.slider = uicontrol('Parent', panel_filt, 'Style', 'slider', ...
                     'Min', 1, 'Max', num_frames, 'Value', 1, ...
                     'Units', 'normalized', ...
                     'Position', [0.05, 0.2, 0.9, 0.3], ...
                     'Callback', @(s,e) sliderMove(s,e,h));

uicontrol('Parent', panel_filt, 'Style', 'text', ...
          'String', 'Arrows to step | Click traces to jump', ...
          'Units', 'normalized', 'Position', [0.4, 0.6, 0.5, 0.3], ...
          'BackgroundColor', 'w', 'HorizontalAlignment', 'right');

% Store data and set up keyboard handler
guidata(h, V);
set(h, 'KeyPressFcn', @(s,e) keyPressHandler(s,e,h));

fprintf('✓ Viewer ready!\n');
```

========== CALLBACK FUNCTIONS ==========

```
function updateFrame(h, frame_idx)
    V = guidata(h);

    % Validate bounds
    frame_idx = max(1, min(round(frame_idx), V.num_frames));

    % Read frame from disk
    try
        frame_data = V.mObj.reconstructed_movie(:,:,frame_idx);

        % Apply polarity correction
        if strcmp(V.polarity, 'FLIPPED')
```

```
            frame_data = -frame_data;
        end

        % Apply spatial blur if enabled
        if get(V.chk_blur, 'Value')
            frame_data = imgaussfilt(frame_data, 1.5);
        end

        % Update display
        set(V.img_h, 'CData', frame_data);
        set(V.title_h, 'String', sprintf('Frame %d / %d | Time: %.3fs', ...
            frame_idx, V.num_frames, V.time_axis(frame_idx)));
        set(V.slider, 'Value', frame_idx);

        % Update all trace markers
        t_val = V.time_axis(frame_idx);
        set(V.xline_main, 'Value', t_val);

        num_plots = V.grid_rows * V.grid_cols;
        for i = 1:num_plots
            set(V.grid_xlines(i), 'Value', t_val);
        end

        V.curr_frame = frame_idx;
        guidata(h, V);

    catch ME
        set(V.title_h, 'String', sprintf('Read Error: %s', ME.message));
    end
end

function sliderMove(src, ~, h)
    val = round(get(src, 'Value'));
    updateFrame(h, val);
end

function traceClick(~, event, h)
    V = guidata(h);
    t_click = event.IntersectionPoint(1);
    [~, frame_idx] = min(abs(V.time_axis - t_click));
    updateFrame(h, frame_idx);
end

function keyPressHandler(~, event, h)
    V = guidata(h);
    current = V.curr_frame;

    switch event.Key
        case 'rightarrow'
            target = current + 1;
        case 'leftarrow'
            target = current - 1;
        case 'uparrow'
            target = current + 10;
```

```
        case 'downarrow'
            target = current - 10;
        otherwise
            return;
    end

    updateFrame(h, target);
end
```

Published with MATLAB® R2024b